Lab W11: 2-3 Tree Insertion

Growing Pains

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

Overview

In lecture you built 2-3 trees by hand. In this lab you teach a program to do the same thing. You are given a working 2-3 tree that can already search for a value, list its values in order, and print itself level by level. What it cannot do yet is insert. Your job is to write insertion, including every kind of split you drew in lecture.

The work is broken into four checkpoints. Each checkpoint has a short test that is already in the driver, along with the exact output it should print, so you can tell when one checkpoint works before you move on to the next. Points are awarded per checkpoint.

The Starter Code

The project contains four pieces. Only one of them needs your code.

- `TwoThreeNode` (complete). One node of the tree. Besides the usual getters, it gives you these helpers:
  - `isLeaf()` and `isFull()`. A node is full when it already holds two values.
  - `addValue(value, rightOfValue)`, which adds a value to a node that is *not* full and keeps the values in order. The node you pass as `rightOfValue` becomes the child just to the right of the new value. Pass `null` (Java) or `nullptr` (C++) for a leaf.
  - `setValue`, `setNumValues`, and `setChild`, for rebuilding a node by hand during a split.
- `SplitResult` (complete). What a split hands back to the level above it: the value that moves up, and the new node that belongs to the right of that value.
- `TwoThreeTree` (three methods to write). Search (`contains`), the inorder traversal, and `printTree` are done. You write the three insertion methods marked `TODO`. Read the comment above each one; it tells you what the method must do and what it must return.

- **The driver** (`TreeDriver.java` for Java, `main.cpp` for C++, complete). It runs the four checkpoint tests and then the three example sequences. You may add your own tests at the bottom of `main`.

Use the insertion algorithm from our lectures and book. You may add private helper methods to `TwoThreeTree` if they help, but do not change `TwoThreeNode`, `SplitResult`, or the methods that are already complete.

The Three Methods You Write

- **`insert(value)`**, the public method. It handles what happens only at the top of the tree, and it calls the recursive method to do the rest.
- **`insert(node, value)`**, the recursive method. It works its way down to the leaf where the value belongs, and on the way back up it deals with any split that happened below it.
- **`split(node, value, rightOfValue)`**. It splits a node that is already full and must take in one more value, for a leaf or for an internal node.

The only comment you need to write is the header comment at the top of `TwoThreeTree.java` (Java) or `TwoThreeTree.cpp` (C++). It is already set up for you; fill in your name after `@author`.

Checkpoints

Work through the checkpoints in order. All four use the same three methods: write just enough to pass Checkpoint 1, run the driver, then extend the same code for Checkpoint 2, and so on. You submit one finished project, and I grade each checkpoint by running your final code. Your output must match the expected output exactly. Each checkpoint also prints the inorder traversal; if that line is not in sorted order, something is wrong even if the levels look reasonable.

Checkpoint 1: Inserting without a split

Insert into an empty tree, and into a leaf that still has room.

```
Checkpoint 1: insert 50 30
Level 1: [30 50]
Inorder: 30 50
```

Checkpoint 2: Splitting the root

The root is a full leaf, and one more value arrives.

```
Checkpoint 2: insert 50 30 70
Level 1: [50]
Level 2: [30] [70]
Inorder: 30 50 70
```

Checkpoint 3: Splitting a leaf whose parent has room

```
Checkpoint 3: insert 50 30 70 10 20
Level 1: [20 50]
Level 2: [10] [30] [70]
Inorder: 10 20 30 50 70
```

Checkpoint 4: Splits that travel up the tree

A split can make the parent overflow, so the parent splits too, and so on up to the root. This test uses the insertion example from lecture, so you can compare the result with the lecture drawings.

```
Checkpoint 4: insert 42 36 16 14 32 31 40 8 21 34 20 22 50 15
Level 1: [16 31]
Level 2: [14] [21] [36 42]
Level 3: [8] [15] [20] [22] [32 34] [40] [50]
Inorder: 8 14 15 16 20 21 22 31 32 34 36 40 42 50
```

Example Sequences

After the checkpoints, the driver runs three example sequences and prints the tree after every single insertion. No expected output is given for these. Once Checkpoint 4 passes, check a few of the steps against your own hand drawing.

- **Sequence A:** 11 39 61 96 74 9 24 67 69 64 52 65 82 60
- **Sequence B:** 46 80 88 13 35 91 23 6 57 56 90 53 99 81
- **Sequence C:** 98 7 75 33 45 19 38 4 62 66 1 76 86 59

Optional Challenge: 2-3-4 Tree

Add a second class, `TwoThreeFourTree`, that inserts into a 2-3-4 tree using **preemptive splitting**. You will need your own node class with room for three values and four children. Add a test to the driver that runs Sequence A through both trees and prints both results. This challenge is not graded. Try it if you want to see how the same splitting ideas carry over to a 2-3-4 tree.

Grading

Table 1: Points for Lab W11.

Item	Points
Checkpoint 1: inserting without a split	5
Checkpoint 2: splitting the root	10
Checkpoint 3: splitting a leaf whose parent has room	10
Checkpoint 4: splits that travel up the tree	25
Total	**50**

Project Setup and Submission

You have been given a folder named `labw11` with the project already set up: for Java, this is an Eclipse project with the package structure already in place; for C/C++, this is the source layout together with the `Makefile`. Do your work inside this folder as given. Do not rename it or rearrange its structure, the layout is part of what gets submitted.

If you are working in Java, submit the entire `labw11` Eclipse project, not just the individual `.java` files.

If you are working in C/C++, submit the entire `labw11` folder, including the `Makefile` and the rest of the project layout, not just `TwoThreeTree.cpp` on its own.

Put the `labw11` folder in your GitHub repository and commit it.